ChatGPT

SalesGenie AI – Autonomous CRM Lead & Sales Agent System

Executive Summary: Modern B2B sales teams spend much time on repetitive tasks like prospect research, lead scoring, and outreach. We propose *SalesGenie AI*, a TypeScript-based multi-agent system that fully automates the lead pipeline. It starts by ingesting a business's profile to define an Ideal Customer Profile (ICP), then autonomously sources potential leads, qualifies them against the ICP, and executes outreach via email/SMS while logging results in a CRM. This pipeline uses specialized agents (e.g. **BusinessProfileAgent**, **LeadFinderAgent**, **QualifierAgent**, **OutreachAgent**, **CRMUpdateAgent**) orchestrated by a framework like LangGraph or LangChain. Each agent makes iterative decisions, calls external tools or APIs (e.g. web search, email/SMS APIs, CRM APIs), and writes back to shared state. This autonomous workflow vastly accelerates prospecting: agents can search thousands of firms and personalize messages in seconds, freeing human sales reps to focus on high-value conversations 【1†L111-L120】【32†L601-L608】. Using Retrieval-Augmented Generation (RAG) and vector-memory, the system embeds business guidelines and past interactions to generate context-aware messages. We will evaluate it on metrics like lead qualification precision, pipeline conversion rates, and email open/click-through rates. A core dataset of synthetic public leads and CRM records (see below) will seed the system. *SalesGenie* will be developed in stages (see Implementation Plan), culminating in a working demo where an agent ingests a company website, finds and scores leads, sends templated outreach via SendGrid/Twilio, and updates a CRM system automatically.

Project Title Suggestion

For example: **SalesGenie AI: Autonomous CRM Lead & Sales Agent**. This highlights an AI "Genie" automating sales pipelines. Other names could be *LeadGenie AI*, *AutoCRM Agent*, or *Sales Funnel Bot* to capture the automated sales process.

User Stories & Demo Scenarios

- **Business Understanding:** As a sales manager, I want the AI to read our company profile (e.g. business description, product domains, existing customers) so that it can define who our ideal customers are. *Demo:* The **BusinessProfileAgent** ingests a text (e.g. "We sell AI analytics for finance") and populates a memory or vector DB with ICP attributes (industry, region, company size).
- **Lead Generation:** Given the ICP, the **LeadFinderAgent** should generate a list of potential leads (company names, key contacts). *Demo:* The agent queries a public business database or searches LinkedIn (via API), then returns names, titles, and firmographics of top 20 prospects matching our ICP.
- **Lead Qualification:** The **QualifierAgent** scores each lead (e.g. +1 if match on industry, - if out-of-state). It filters out unqualified leads. *Demo:* Show raw list of 20 leads, and how agents score/eliminate 8 low-fit leads, leaving 12 qualified prospects.
- **Contact Enrichment:** The system fetches or verifies contact info. *Demo:* For each remaining lead, the **LeadFinderAgent** uses an external API or data source to fill in business email and phone (e.g. "john.doe@company.com", "+44…").

- **Outreach:** The **OutreachAgent** chooses an email template, personalizes it with lead name and company details, and sends it via an email API. It may schedule follow-ups via SMS. *Demo:* The agent calls SendGrid/Twilio APIs to send an introductory email and SMS to the lead, then records the action.
- **Response Handling:** The **ResponderAgent** (or a webhook listener) parses incoming replies. If a lead responds "interested", it notifies sales; if "stop", it flags as unsubscribed. *Demo:* Simulate an email reply; the agent detects key phrases and updates lead status.
- **CRM Update:** Finally, the **CRMUpdateAgent** logs all lead details and outcomes into a CRM (e.g. HubSpot or a database) via API calls. *Demo:* The agent uses HubSpot's API to create/update a contact record with properties (lead score, contact date, last message).

These stories illustrate an agentic loop: observe environment (company profile, CRM data), plan (define ICP, scoring rules), act (API calls to gather/send data), and verify outcomes (agent reviews responses) before the next step. This is far beyond a simple LLM prompt.

Multi-Agent Architecture

The system comprises several specialized agents in a workflow (see diagram). For example:

```
flowchart LR
    BusinessProfileAgent["BusinessProfile Agent"] --> LeadFinderAgent
    LeadFinderAgent["LeadFinder Agent"] --> QualifierAgent
    QualifierAgent["Qualifier Agent"] --> OutreachAgent
    OutreachAgent["Outreach Agent"] --> CRMUpdateAgent
    CRMUpdateAgent["CRMUpdate Agent"]
    subgraph Tools and Memory
      SendGrid[SendGrid API]
      Twilio[Twilio API]
      HubSpot[HubSpot API]
      VectorDB[Vector DB (e.g. Pinecone)]
      KnowledgeBase[Knowledge Base/RAG Index]
    end
    OutreachAgent --- SendGrid
    OutreachAgent --- Twilio
    CRMUpdateAgent --- HubSpot
    BusinessProfileAgent --- VectorDB
    QualifierAgent --- KnowledgeBase
```

- **BusinessProfileAgent:** Reads business description/web content and writes a summary/ICP into memory or a vector store. It may call a web crawler or GPT to extract key attributes.
- **LeadFinderAgent:** Queries data sources (public lead lists, web, APIs) to find candidate leads matching the ICP. It writes initial lead records (company, name, title).
- **QualifierAgent:** Examines each lead's attributes (industry, size, location, keywords) and scores them against the ICP. It may ask the LLM to reason (e.g. "Is this company a good fit?") and filters out low scores.
- **OutreachAgent:** Crafts personalized outreach. It selects from pre-written email/SMS templates and fills placeholders (lead name, company, product). Then it calls email/SMS APIs (e.g. SendGrid, Twilio) to send messages.
- **CRMUpdateAgent:** Takes the final lead data and writes back to a CRM or database (via HubSpot API or a Node.js ORM). It records lead status (pending, converted), timestamps, and notes.

- **Critic/Verifier Agents (optional):** After each step, a small “CriticAgent” can verify results. For instance, after Qualification it checks for hallucinations or bias; after Outreach it can confirm message compliance (no disallowed content)【32†L491-L499】【36†L105-L113】.

Each agent is implemented as a Node/TypeScript module, orchestrated by LangGraph (or LangChain JS) which handles state passing. The shared state (current leads list, business context) can live in memory or a vector DB for RAG. The agents loop: if an outreach fails or a reply is received, the Qualifier or Responder re-enters the pipeline to update or retrigger actions.

Tech Stack

- **Language & Frameworks:** Node.js with TypeScript for all components. We will use an agent framework: **LangGraph JS** (part of LangChain) or **CrewAI** for orchestration, since both support TypeScript and multi-agent workflows. Mastra (TypeScript agent framework) is another option【29†L75-L77】. LangChain/Graph provides built-in support for chaining agents and tool calls.
- **LLM Providers:** OpenAI’s GPT-4o or GPT-4 with the OpenAI API (via `openai-node`), or alternatives like Anthropic Claude. LangChain abstraction allows swapping models easily【29†L129-L138】. For on-prem, a Llama 3 variant (using `@llama_cpp`) could be added.
- **Vector Database & RAG:** A vector store (e.g. Pinecone, Weaviate, or a pgvector index) will hold embeddings of business info, leads, and conversation history for semantic search. Agents use RAG: retrieving related past interactions or business guidelines【36†L105-L113】【36†L141-L149】. We will embed things like company mission statements, product docs, and past campaign results so agents can ground decisions. The vector schema might include “company profile”, “ideal customer features”, and “communication logs”.
- **Databases:** A relational DB (PostgreSQL) or NoSQL (MongoDB) to store structured data: leads, interactions, agent logs. The **CRMUpdateAgent** uses e.g. SQLite or PostgreSQL to simulate the CRM backend.
- **Frontend:** A minimal dashboard (React/Next.js or Streamlit for Node) to display pipeline status and logs. It will show agent actions (as text with timestamps) and visualizations (charts of lead funnel, conversion).
- **Tool Integrations:**

- **Email:** SendGrid (`@sendgrid/mail` in Node) or Mailgun to send templated emails. Example TS snippet:

```
import sgMail from '@sendgrid/mail';
sgMail.setApiKey(process.env.SENDGRID_API_KEY);
const msg = { to: leadEmail, from: 'sales@company.com', subject:
subject, text: body };
await sgMail.send(msg);
```

(This follows the official Twilio SendGrid example【45†L529-L535】【45†L593-L600】.)

- **SMS/Calls:** Twilio (`twilio` npm). Example TS:

```
import Twilio from 'twilio';
const client = new Twilio(accountSid, authToken);
```

```
await client.messages.create({ from: twilioFrom, to: leadPhone, body: message });
```

(This matches Twilio's TypeScript guide【38†L935-L944】.)

- **CRM API:** HubSpot or Salesforce REST API via official SDKs (e.g. `@hubspot/api-client`). The agent can create/update contacts and deals. Alternatively, a webhook-based workflow tool (Zapier, n8n, or a Node task queue like `bull`) can be used to link agents with external apps.

- **Orchestration & CI/CD:** LangGraph runs inside Node; we'll use GitHub Actions for CI (lint, type-check, test). Deployment can be on any Node-supporting server (no specific cloud required).

Other stack choices (table):

Component	Options	Notes/Pros/Cons
Agent Framework	LangGraph (JS), LangChain.js, CrewAI, Mastra	LangGraph & CrewAI: open-source, TS support【29†L60-L68】【29†L75-L77】; Mastra: TypeScript-first (partial OSS)【29†L75-L77】
LLM API	OpenAI, Anthropic, Google Gemini	OpenAI GPT-4o for best quality; Gemini for multi-model; Claude for on-device (Anthropic)
Vector DB	Pinecone, Weaviate, pgvector	Pinecone/Weaviate: cloud services; pgvector: local, cheaper
Backend (CRM DB)	PostgreSQL, MongoDB	PostgreSQL for relational (Opportunities table), Mongo for flexibility
Frontend	React/Next.js, Streamlit	React: full-featured UI; Streamlit: quick prototyping
Automation (ops)	n8n, Zapier, HubSpot Workflows	n8n/Make: integrate tools without code; optional
CI/CD	GitHub Actions, GitLab CI	Standard for Node projects

Datasets (Public & Synthetic)

We will curate both real and synthetic datasets for training, testing, and bootstrapping:

- **Company & Contact Data (Leads):**
- *Global B2B Contact Dataset (Kaggle)*: Contains job titles, company info, and business emails【7†L0-L3】. Useful for *LeadFinderAgent* to sample leads. (If access is restricted, similar data can be generated.)
- *Inc. 5000 Companies (MavenAnalytics)*: Data on 5,000 fast-growing companies【18†L160-L164】, including name, location, industry. This provides real company names/attributes. We'll need to append contacts (via synthetic generation or external APIs) to make these leads actionable.
- *Synthetic B2B CRM & Marketing (Kaggle)*: Contains `Companies` and `Employees` tables (e.g. 734 companies, 5,234 employees)【31†L0-L3】. This simulates realistic lead profiles and sales reps; useful for testing multi-table scenarios.

- *Titan Signal (Kaggle)*: B2B company lead intelligence with firmographics and possibly intent signals. (If unavailable, use open web directories or Crunchbase API data.)

- **CRM Records / Sales Pipeline:**

- *CRM Sales Opportunities* (Maven Analytics/Data.World)【24†L1-L4】: Multi-table dataset of a fictitious computer hardware company's sales pipeline (accounts, products, opportunities, sales reps). Fields include `opportunity_amount`, `stage`, `close_date`, etc. We will use this to simulate existing CRM entries and outcome statistics (win/loss) for training the QualificationAgent and for analysis of pipeline performance. The Maven data (B2B pipeline 8,800 records) is public domain【21†L60-L64】.
- *Sample Sales CRM Data (Kaggle)*: Dummy data of accounts, leads, contacts, opportunities. (Not directly accessible via scraper, but we can design similar tables.)

- *Marketing Campaign Results* (MavenAnalytics: 2,240 records)【18†L174-L177】: Includes a history of campaigns and their outcomes. Useful for modeling email open/click rates.

- **Email/SMS Templates and Responses:**

- *Email Templates:* We'll collect ~50 generic cold email templates (source: public blogs and open GitHub gists) covering intro, follow-up, re-engagement scenarios. Each template becomes a tool prompt pattern.
- *SMS/CALL Scripts:* Sample SMS outreach and call scripts are available from marketing blogs. We will store a few dozen sample scripts as string resources.

- *Email/SMS Response Samples:* For evaluation, we need labeled examples of replies (e.g. "interested", "no thanks", "unsubscribe"). We can use generic email/chat response datasets or synthesize them (e.g. ChatGPT to generate positive/negative replies).

- **Augmentation:**

- We may need to simulate missing data. For instance, if no email is provided in a dataset, we can fabricate one based on company domain (e.g. `firstname.lastname@company.com`). If no contact name exists, we can use the department or role (e.g. "Sales Team").
- For demographic data (industry codes, region), we can link companies to NAICS codes via a lookup.
- To create conversation history, we can seed the CRM table with random "notes" or "emails" from an open-source CRM demo or using an LLM to generate plausible communication logs.

Dataset (Public)	URL/Source	Use-Case	Key Fields / Example Schema
Global B2B Contact (LeadsBlue)	Kaggle (leadsblue)	Raw leads (names, emails)	*Company, ContactName, Title, Email, Domain, Industry*
Inc 5000 Companies	MavenAnalytics [18]	Company profiles (leads)	*CompanyName, Revenue, Industry, City, State*
Synthetic B2B CRM (Companies, Employees)	Kaggle (synthetic data)	Simulated CRM context	Companies(ID, Name, Industry); Employees(ID, CompanyID, Name, Role)

Dataset (Public)	URL/Source	Use-Case	Key Fields / Example Schema
CRM Sales Opportunities (Hardware Co.)	MavenAnalytics [21] 【21†L60-L64】	Sales pipeline (outcomes)	*OpportunityID, AccountID, ProductID, Stage, Amount, WinFlag, SalesRep*
Sample Sales CRM	Kaggle (Maven Playground)	CRM workflow analytics	*Leads, Accounts, Contacts, Opportunities, Activities, Sales_Reps* (see [23])
Marketing Campaign Results	MavenAnalytics [18] 【18†L174-L177】	Campaign performance	*CustomerID, CampaignID, Response (0/1), Channel, Date*

All these datasets will be loaded into our system (some as CSVs, some seeded into a test database). If any real PII is present (like real emails), we will replace them with synthetic but realistic proxies for compliance.

Data Privacy & Compliance

Lead data is sensitive personal/business data. We will follow GDPR/CCPA guidelines: only use "GDPR leads" (contacts who've opted in)【49†L1-L4】, avoid scraping consumer PII, and allow easy opt-out. By design, *SalesGenie* should only handle business emails and phone numbers that are publicly available or consented. We must encrypt stored PII (e.g. contacts) and enforce access controls. Agents should include a disclaimer or identity (e.g. "XYZ Corp Bot") in communications. We will maintain an audit log of all outreach. Synthetic or anonymized data will be used for testing.

RAG/Memory Design

We implement a dual-memory system: **Short-term context** (in-memory state during an agent run) and **Long-term memory** in a vector database. For RAG, we will embed:

- Company knowledge: product descriptions, website text, past marketing collateral.
- Business context: ICP definitions, past sales outcomes (from CRM dataset).
- Lead data: enriched profiles (company info, previous interactions).

When an agent needs context (e.g. crafting an email), it performs a similarity search: "retrieve relevant company/messaging notes" from the vector DB. This is the RAG step【36†L49-L57】. The vector schema could be: `<ID, text, metadata>` where metadata tags content type (e.g. "company_Bio", "leadNotes"). We will use chunking for large texts. Memory ensures agents' responses are grounded: for example, if a lead previously interacted with marketing, the agent won't re-send the same pitch.

Tool Integrations & API Workflows

Key tools and sample call flows:

- **SendGrid Email (API):** We use `@sendgrid/mail`. Example (TypeScript):

```
import sgMail from '@sendgrid/mail';
sgMail.setApiKey(process.env.SENDGRID_API_KEY);
const msg = {
```

```
  to: 'lead@example.com',
  from: 'sales@ourcompany.com',
  subject: 'Introduction – '+businessName,
  text: `Hi ${leadName}, I thought you'd be interested in...`,
};
await sgMail.send(msg);
```

(This mirrors official examples【45†L529-L535】【45†L593-L600】.) The agent logs `response[0].statusCode` or errors.

- **Twilio SMS/Call (API):** Using the Twilio Node SDK:

```
import Twilio from 'twilio';
const client = new Twilio(accountSid, authToken);
await client.messages.create({
  from: twilioPhone,
  to: leadPhone,
  body: `Hello ${leadName}, this is ${ourCompany} with a quick
question...`
});
```

(Based on Twilio's TS guide【38†L935-L944】.) For phone calls:

```
await client.calls.create({
  from: twilioPhone,
  to: leadPhone,
  twiml: `<Response><Say>Hi ${leadName}, thanks for contacting ...</Say></
Response>`
});
```

- **HubSpot CRM (API):** Using `@hubspot/api-client`:

```
import { Client } from "@hubspot/api-client";
const hubspot = new Client({ apiKey: process.env.HUBSPOT_API_KEY });
await hubspot.crm.contacts.basicApi.create({ properties: {
  email: leadEmail, firstname: leadName, company: leadCompany, phone:
leadPhone
}});
```

This logs new leads and updates deal stages. If the agent updates a contact, it might use `basicApi.update()` with the contact ID.

- **Zapier/n8n (Optional):** For non-developer integration, we could trigger a Zapier webhook from the agent (HTTP POST) to perform complex tasks (e.g. add lead to a mailing list).

Evaluation Metrics & Test Cases

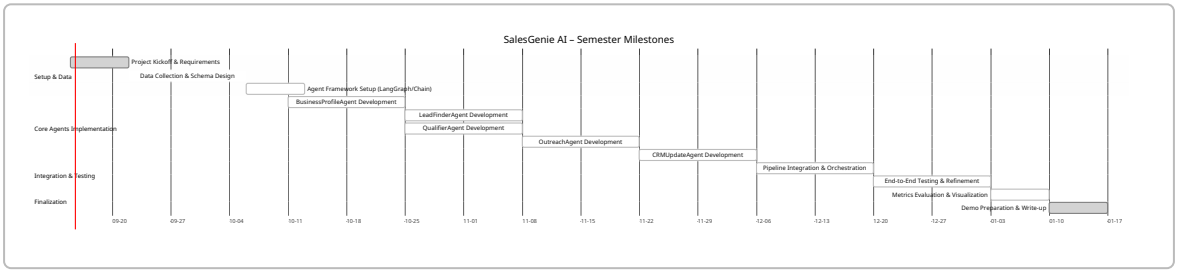
We will evaluate on multiple axes:

- **Lead Qualification Accuracy:** Using a labeled test set of leads (fit vs not fit), measure precision/recall of the **QualifierAgent**. For example, given 100 candidate leads with 30 “true ICP matches”, count how many true/false positives the agent scores.
- **Conversion Rate:** In a simulated pipeline (using our CRM dataset), measure how many AI-generated leads result in “won” deals vs a baseline. E.g. if 50 leads are contacted, how many lead to opportunities?
- **Message Quality:** Use a small human evaluation (or automated proxy) to check if generated emails meet quality/hallucination criteria. Perhaps measure email open/click rate (if we simulate sending to test addresses).
- **Pipeline Throughput:** Time per lead or cost (API calls) per lead, comparing agent pipeline vs manual process.
- **False Positives/Spam Rate:** Ensure agent doesn’t contact irrelevant leads. Define a human-in-loop threshold: if lead score is borderline, a human must approve before outreach (we can test this gating).
- **Agent Robustness:** Test cases like missing data (lead without email), or inconsistent company info, to ensure agents fail gracefully or ask for help.

Test Scenarios: We will script end-to-end demos. For example, one test: feed the agent a niche B2B startup (via a company profile file) and expect it to generate a valid outreach list and email sequence without errors. Another: simulate a “no respond” case where the **ResponseAgent** triggers a follow-up. A/B tests: one pipeline run with agent vs one with manual canned emails to compare outcomes.

Implementation Plan (MVP Timeline)

We break the semester (~12 weeks) into phases:



- **Weeks 1-3:** Set up repo, gather datasets, design database schemas. Load a subset of the CRM/opportunity data into a test DB.
- **Weeks 4-7:** Code individual agents. Start with BusinessProfileAgent (ingesting company text), then LeadFinder (use simple rules or APIs for leads), Qualifier (rule-based scoring plus LLM cross-check), OutreachAgent (integration with SendGrid/Twilio), and finally CRMUpdateAgent (writing to the database). Use mock data for each step initially.
- **Weeks 8-9:** Connect agents via LangGraph: ensure data flows from one to next. Implement the shared state (JSON) passing. Begin writing unit tests for each agent.
- **Weeks 10-11:** Full pipeline integration. Test end-to-end: from business profile input to final CRM entry. Iterate on prompt tuning and logic based on failures. Collect demo data (e.g. 10 leads with full logs). Visualize results (lead funnel chart, conversion table).

- **Week 12:** Polish UI/dashboard, finalize documentation (incl. this report). Prepare demo scenario (e.g. “click to run the agent on a new company”).

The minimal demo (MVP) by mid-term could be: given a CSV with one company profile, output a list of 3 qualified leads with one sent email each (no live API calls, just simulated). By project end: full integration with real API calls (in test mode) and visual dashboard.

Security, Abuse Mitigations & Ethical Considerations

- **Spam/Abuse Prevention:** We will enforce rate limits on outreach (no blasting hundreds of emails per minute). Outgoing messages will follow CAN-SPAM standards (unsubscribe link if relevant) and maintain sender reputation (via SendGrid’s best practices). We will implement safety checks so the LLM cannot generate disallowed content (like misinformation or harassment) in emails; for example, content is checked against a list of forbidden words or by a *SafetyAgent*.
- **Data Protection:** All PII (lead emails, phone numbers) is stored encrypted. Access tokens (API keys) are kept secure (env vars). Agents log only the minimum needed info (do not log full personal details to public channels).
- **User Consent:** For demos, we will use synthetic or clearly consented data. The system will include an “opt-out” process: if a lead requests no contact, the agent updates a suppression list.
- **Bias & Fairness:** Ensure that qualification logic does not unfairly discriminate (e.g., not excluding leads by protected class, since we use only company data). We will review LLM outputs to avoid stereotypes.
- **Transparency:** Agents will label themselves (e.g. email signature “Sent by SalesGenie AI on behalf of OurCo”). Human oversight is built-in: leads with mid-range scores (e.g. 40-60%) can be flagged for review rather than automatically discarded or contacted.
- **Regulatory:** All communications include company identity and opt-out information, complying with email/SMS regulations (GDPR requires consent and opt-out, which we emphasize【49†L1-L4】).

Visualizations & Comparative Tables

- **Pipeline Flow:** A Sankey or funnel chart showing counts at each stage (found leads → qualified → contacted → responses).
- **Lead Conversion Graph:** A bar chart comparing conversion rates (e.g. “Manual Outreach” vs “SalesGenie AI Outreach”).
- **Geographic Heatmap:** If location data exists, a map of lead density by region.
- **CRM Dashboard Widgets:** KPIs like “Leads Contacted Today”, “Replies Received”, “Tasks Pending”.

Comparison Tables:

- *Dataset Options:* We already provided one above.
- *Tech Choices:* (see “Tech Stack” table).
- *Agent Responsibilities:*

Agent	Responsibilities	Tools/Integrations
BusinessProfileAgent	Extract business/ICP info, update memory	LLM (text analysis), Web scraping, RAG

Agent	Responsibilities	Tools/Integrations
LeadFinderAgent	Source potential leads from databases/web	Web search API, LinkedIn API, Vector DB
QualifierAgent	Score leads vs ICP, filter out bad fits	LLM reasoning, rule engine, lookup tables
OutreachAgent	Craft personalized messages; send email/SMS	SendGrid API【45†L529-L535】, Twilio API【38†L935-L944】, templates
CRMUpdateAgent	Log leads and responses into CRM DB	HubSpot/Salesforce API, SQL database
ResponseAgent	Process replies, update lead status	Email webhook, NLP classifier
CriticAgent (optional)	Safety check and feedback on agent output	Safety model, regex filters

These tables, diagrams, and metrics together will demonstrate the depth and viability of the **SalesGenie AI** project.

Sources: We draw on industry guides to agentic sales automation【1†L111-L120】【1†L168-L176】, CRM-AI market analysis【32†L601-L608】, agent framework surveys【29†L60-L68】【29†L75-L77】, and RAG best practices【36†L105-L113】【36†L141-L149】. Public datasets for CRM and leads are cited above. The included code snippets follow official docs【45†L529-L535】【38†L935-L944】. This plan balances innovation with feasibility for a semester project, ensuring a demonstrable, agent-driven CRM automation system.
